Lecture Slip Questions: Answers

CSCI 235 Software Design and Analysis II, Hunter College
Lecture slip of Tuesday, November 3, 2026 (Operator Combinations, Unique Strings, Task B, and Expression Trees)

Thank you for the comments on your slips. They are answered below. The table and the table of contents are links: click an entry to jump to its answer.

Your comment or question	Answer
Go over today's topic again next class.	Section 1
What does using the different orders with an expression do? Why use one order over another?	Section 2
I understand recursion and base cases, but I have trouble writing the recursive call that makes progress toward the base case.	Section 3
Please do not do a mid-class quiz. / The optional quiz would be very helpful!	Section 4
We can't see what is being pointed at until it is drawn.	Section 5

Contents

1 Today's topic again, in one page **2**

2 What do the different orders do, and why choose one? **2**

3 Writing the recursive call that makes progress **4**

4 The mid-class quiz **5**

5 Seeing what is being pointed at **5**

1 Today's topic again, in one page

Today had three ideas. Each one is the previous one with a picture added.

Idea 1: a search that grows an expression. We fill the gaps between numbers with an operator and recurse. The state is the string so far (`expr`) and `k`, how many numbers are already used. Every call does **choose, explore, undo**: append an operator and the next number, call with `k + 1`, then cut the string back with `expr.resize(len)`.

Idea 2: do not repeat yourself. `printAll("ABB")` treats the two B's as different people, so it prints 6 lines but only 3 different strings. Keeping a `std::set<char> tried` *inside* each call lets that seat skip a letter it has already tried: 3 lines and 9 calls instead of 6 lines and 16 calls. For Make 24, Task B is Task A plus one change: instead of returning `true` or `false`, a call returns *the first expression it finds*, and the empty string means "none here".

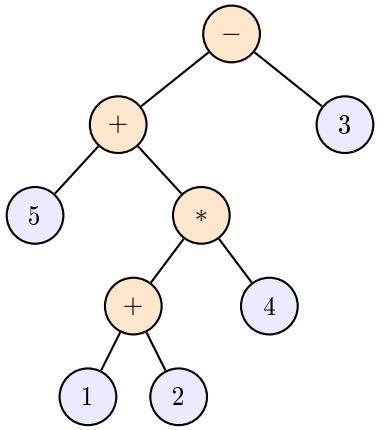
Idea 3: an expression is a tree. Every operator has two subtrees, so `(3 + 4) * 2` is a tree whose **root** is the *last* operation performed. Reading the tree in three different orders gives three different texts of the same expression (Section 2).

If you remember one thing...	**...remember this**
Choose, explore, undo	Whatever you change before the recursive call, you restore after it.
Pruning with `tried`	Skip a branch that can only repeat a result you already have.
Task B	Same search as Task A; the answer carried back up is a string, not a bool.
Trees	The root is the last operation; the leaves are the numbers.

2 What do the different orders do, and why choose one?

Short answer. The three orders are three ways to write down the *same* tree. Each one is useful for a different job, and the job decides the order: **do you need the children's answers first, or the parent's first?**

Take the tree for `5 + (1 + 2) * 4 - 3`. It is built "first operation first": $1+2$, then $\times 4$, then $5+$, then -3 last, so the last operation, the `-`, is the root.



Order	Rule	Text for this tree
Pre-order	root, left, right	`- + 5 * + 1 2 4 3`
In-order, with parentheses	left, root, right	`((5 + ((1 + 2) * 4)) - 3)`
Post-order	left, right, root	`5 1 2 + 4 * + 3 -`

An analogy: the size of a folder on your computer. A folder contains files and other folders. To compute its total size you must know the size of every subfolder first, and only then can you add them up. That is **post-order**: the children are finished before the parent. To *copy* the folder structure you do the opposite: create the folder first, then fill it. That is **pre-order**: the parent comes first. Neither order is "right"; each fits its job.

What each order is good for.

- **In-order** is the order people read. With parentheses it is exactly what we write on paper. *Without* the parentheses it is ambiguous: `5 + 1 + 2 * 4 - 3` reads as 11, but the tree evaluates to 14. The parentheses are what remember the shape of the tree.
- **Pre-order** puts the operator first, so you know what to do before you meet the operands, like a function call: `+(3, 4)`. It is the natural order for *copying* or *saving* a tree, because the parent is written before its children and the tree can be rebuilt from the list.
- **Post-order** puts the operator after its operands. A machine can then evaluate the expression with a stack, left to right, with *no parentheses and no precedence rules.* This is why it matters for the next class.

Post-order evaluation, by hand. Read `5 1 2 + 4 * + 3 -` left to right. A number is pushed. An operator pops two values (the first one popped is the *right* operand), applies the operator, and pushes the result.

Token	Stack (top on the right)	Token	Stack (top on the right)
`5`	`[5]`	`4`	`[5, 3, 4]`
`1`	`[5, 1]`	`*`	`[5, 12]`
`2`	`[5, 1, 2]`	`+`	`[17]`
`+`	`[5, 3]`	`3`	`[17, 3]`
		`-`	`[14]`

So which order do I use? Ask what each node needs. If it needs the answers of its children (evaluate, count, delete), use post-order. If it must be recorded or announced before its children (copy, save, print as a function call), use pre-order. If a person has to read it, use in-order with parentheses. In Class 18 we use post-order text as the *target* of the shunting-yard algorithm: it converts the in-order text people write into post-order text a stack can evaluate.

3 Writing the recursive call that makes progress

You already understand the hard conceptual part: the base case. The skill you are asking about is the other half, and it has a method. **A recursive call must be on a smaller problem, and "smaller" must be something you can name.**

An analogy: going down stairs. The base case is the ground floor. Each call must take you down at least one step. Calling the function with the same arguments is standing still (infinite recursion). Calling it with a bigger argument is walking up (it never reaches the ground). So the question to ask before writing the call is: *what number is my staircase, and does this call make it smaller?*

A five-step recipe.

1. **Say what the function promises.** One sentence: "`ops(v, k, expr)` prints every way to fill the gaps from position `k` on."
2. **Name the staircase** (the measure that shrinks). In `ops` it is the number of numbers still unused, $v.size() - k$.
3. **Write the base case:** the step at which the staircase is at the bottom ($k == v.size()$).
4. **Write the recursive call so the staircase gets smaller:** `k + 1`, not `k`.
5. **Trust the call.** Assume the smaller call does its promised job, and write only what *this* level must add (here: append an operator and a number, and undo afterward).

Where the staircase is in the code we have seen.

Function	The staircase	The call that goes down a step
`factorial(n)`	`n`	`factorial(n - 1)`
`printAll(s, k)`	letters not yet seated, $s.size() - k$	`printAll(s, k + 1)`
`ops(v, k, expr)`	gaps not yet filled, $v.size() - k$	`ops(v, k + 1, expr)`
postorder on a tree	height of the subtree	`postorder(node->left)` and `(node->right)`

A small worked example. Write `sumTo(n)`, the sum $1 + 2 + \cdots + n$.

1. Promise: `sumTo(n)` returns $1 + 2 + \cdots + n$.
2. Staircase: `n`. The ground floor is $n = 0$.
3. Base case: `if (n == 0) return 0;`
4. Call that goes down: `sumTo(n - 1)`.
5. Trust it: if `sumTo(n - 1)` returns $1 + \cdots + (n - 1)$, this level only has to add `n`.

```
int sumTo(int n) {
  if (n == 0) { return 0; }       // ground floor
  return n + sumTo(n - 1);        // one step down, then add this level's part
}
```

The three most common mistakes.

Mistake	What happens
`sumTo(n)` (same argument)	Standing still: the calls never end, and the program crashes with a stack overflow.
`sumTo(n + 1)`	Walking up: it never reaches 0.
`printAll(s, k)` instead of `k + 1`	Same as the first: the seat number never advances.

Try it. Write `void countdown(int n)` that prints $n, n-1, \ldots, 1$. Name the staircase first. *Answer:* the staircase is `n`; base case `if (n == 0) return;`; then print `n` and call `countdown(n - 1)`. Moving the print *after* the call prints $1, 2, \ldots, n$ instead; try to explain why using the call stack.

When you are stuck, trace the *smallest* nonempty input by hand (`sumTo(1)`, or `ops` with two numbers). If you can see exactly which call produces which output, the recursive call you need is usually visible.

4 The mid-class quiz

Two of you asked for opposite things, and both are reasonable: a quiz in the middle of the lecture interrupts the explanation, and a quiz also shows you whether you have understood. Here is how it works:

- The 5-minute mini quiz on the slip is **optional** and does **not** count toward your grade. If you skip it, nothing happens.
- It is there only to give you feedback. The feedback may be produced by AI, and AI can make mistakes, so check it against the lecture notes.
- If you do not want it, leave that part of the slip blank. If you want practice, take it at home and compare with the lecture notes.

5 Seeing what is being pointed at

Thank you for saying this. If you cannot see what is being pointed at until it is drawn, you lose the connection between the words and the diagram, and that is exactly the connection a recursion trace depends on. Three changes help:

- The slides already mark the call under discussion with a red box, a red arrow, and numbered badges for the call order. These are on the PDF, so you can follow along on your own screen.
- In class, I will *say* which node or line I mean before I point ("the left child of the plus"), so it does not depend on seeing the pointer.
- If you cannot see something, please call it out in class. It is a good question, not an interruption.